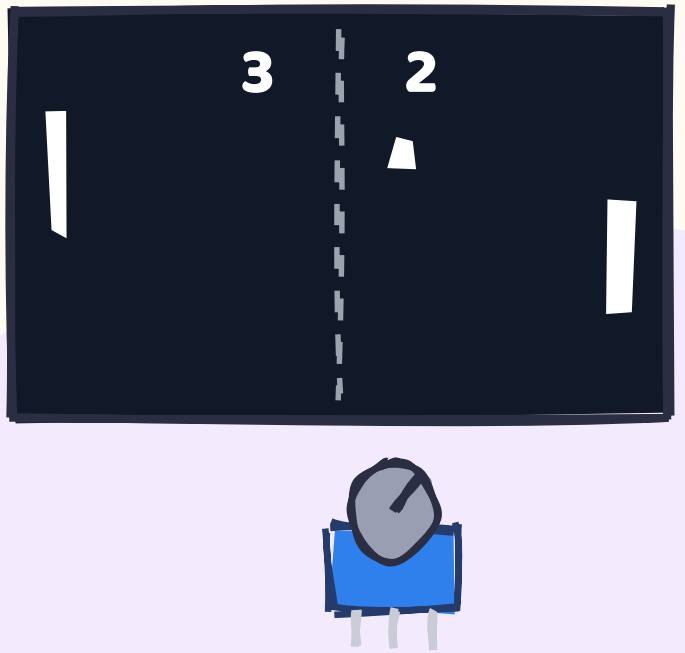
Junior Makers · Class 15

# Pong Game with a Knob Controller

## Build the game AND the controller!

Potentiometer | Mapping numbers | Game loop



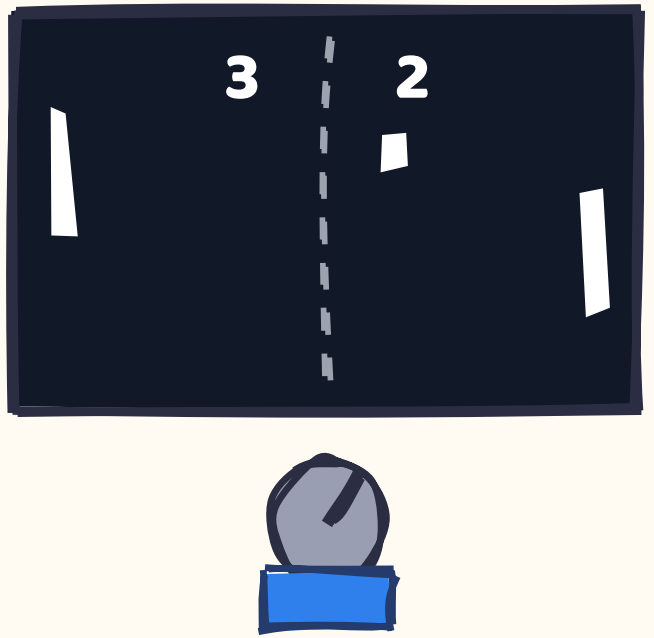

Did you know?
Pong came out in 1972. The first machine stopped working after two days because it was too FULL of coins!

Let's think...

# How can a knob play a video game?

**1972: one knob, one ball**

In the first Pong machines, players turned a simple **knob** to move a paddle up and down. No buttons, no joystick, no touch screen. How does twisting a knob become **movement on a screen**?




Magic wire?

Tiny motor?

A number!

**Today:** we build Pong in **mBlock** and make our own **knob controller** with an Arduino.

Learning objectives

# By the end of this class I can...

| | | |
|---|---|---|
| 1 | **Knob to number:** explain how a **potentiometer** turns a twist into a number from 0 to 1023 | **Proof:** my knob readings at both ends and the middle |
| 2 | **Map:** **map** the knob number to the paddle's position on the screen | **Proof:** turning the knob moves my paddle from top to bottom |
| 3 | **Game loop:** make the ball move and **bounce** using a forever loop and if rules | **Proof:** my ball bounces off the walls and my paddle |
| 4 | **Playtest:** **playtest** my game with a friend and improve it | **Proof:** one change I made after my friend's feedback |

Objective 1 · Knob to number · video

# Video time!




**The Evolution of Video Games: Pong's 50-Year Legacy**

The Sydney Morning Herald · about 7 minutes · youtu.be/763CFRuxovo

## Watch for...

1. What **year** did Pong come out, and where did people play it?
2. How was Pong **controlled**?
3. Name **one way** games have changed since Pong.

Scan to watch

**After the video:** With your partner, list **3 rules** of Pong. A computer has to follow every one of those rules!

Objective 1 · Knob to number

# A potentiometer turns a twist into a number

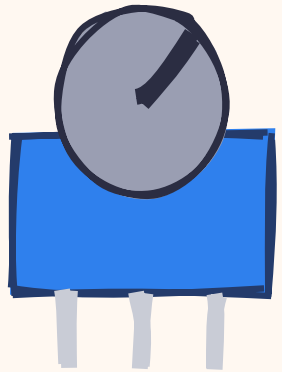
## The idea

Inside the knob is a strip of carbon and a little slider. Turning the knob moves the slider, so the **voltage** on the middle pin changes from **0 V to 5 V**.

The Arduino reads that voltage on pin **A0** as a number from **0 to 1023**. Turn it all the way left: 0. All the way right: 1023.

10k?
10k is the knob's resistance: 10,000 ohms. That is printed on the back.

## Example: a volume knob



0

1023

Turning the volume knob on a speaker or car radio does the same thing. A little twist = a little louder. The knob does not jump between 'on' and 'off': it can be **anywhere** in between. That is called **analog**.

Real world

# Knobs and sliders all around us



**Joysticks**
Two hidden knobs inside: one for left-right, one for up-down



**Car steering**
Sensors tell the car how far the wheel is turned



**Mixing desks**
Sliders set how loud each instrument is



**Oven dials**
Turn to choose the temperature

Find a knob or slider in your classroom. What number could it be controlling?

**Analog vs digital**
A button is digital: only 0 or 1. A knob is analog: any value in between.

Objective 2 · Map

# Mapping: change one range into another

## The idea

The knob gives **0 to 1023**. The paddle must go from **y = -150** (bottom) to **y = 150** (top) on the stage. We need to **map** one range onto the other:

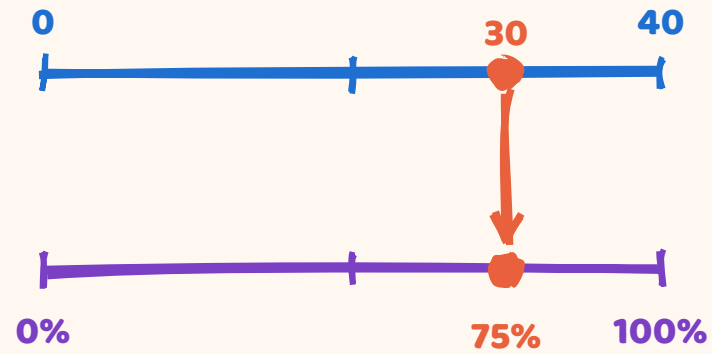
**paddle y = knob ÷ 1023 × 300 - 150**

Knob 0 → -150 (bottom). Knob 1023 → 150 (top). Knob 512 → about 0 (middle).

Shortcut

mBlock and Arduino also have a ready-made map block. Try it after you understand the maths!

## Example: test marks to percent




You scored **30 out of 40** in a test. To get a percentage: 30 ÷ 40 × 100 = **75%**. You mapped the range 0 to 40 onto the range 0 to 100. Same maths, different numbers!

Objective 3 · Game loop

# A game loop: move a little, check, repeat

## The idea

Every game runs a **game loop** many times a second:

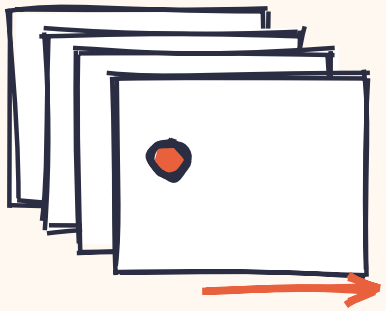
1. **Move** the ball a few steps.
2. **Check** the rules: hit a wall? hit the paddle? missed?
3. **Draw** the new picture.

Then do it all again. Fast loops make smooth movement.

FPS

Most games loop 30 to 60 times every second. That is called frames per second (FPS).

## Example: a flipbook



tiny move each page

Draw a ball on each page of a notebook, moved a tiny bit each time. Flick the pages fast and the ball **moves**! Each page is one turn of the game loop.

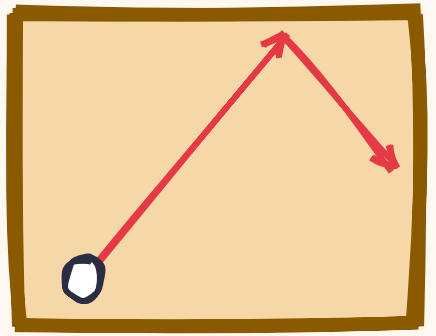
Objective 3 · Game loop

# Bouncing is just an if rule

## The idea

**if** the ball touches the **top or bottom edge** → bounce (Scratch: **if on edge, bounce**).
**if** the ball touches the **paddle** → turn round: **point in direction (0 - direction)**.
**if** the ball goes past the paddle → the other side scores a point and the ball goes back to the middle.

## Example: a carrom striker



same angle in, same angle out

Flick the striker against the side of the carrom board. It bounces off at the **same angle** it came in. Our ball follows that rule too.

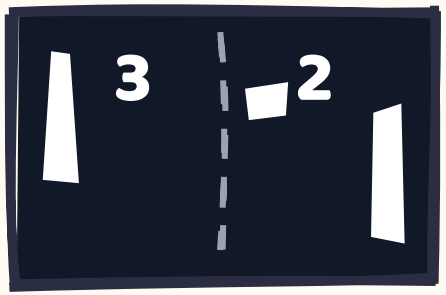
Quick quiz

# Fill the gaps

| Question | Answer |
|---|---|
| 1. The Arduino reads the knob as a number from 0 to ____. | |
| 2. A knob is analog; a button is ____. | |
| 3. Knob 0 → paddle y = ____ (top or bottom?) | |
| 4. Move, check, draw, repeat is called the game ____. | |
| 5. Testing a game with real players is called ____. | |

**Answers**

Cover this! 1: 1023 · 2: digital · 3: -150, bottom · 4: loop · 5: playtesting



Part 2 · Make it

# Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

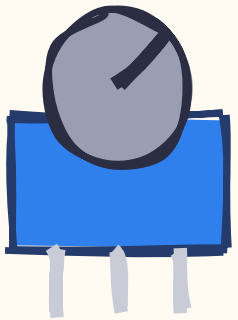
1. Read your knob
2. Human Pong
3. Our Pong kit (per group)
4. Wire the controller
5. Make the paddle follow the knob
6. Code the ball
7. Serve, score and sound

MAKE

Objective 1 · Knob to number · test it

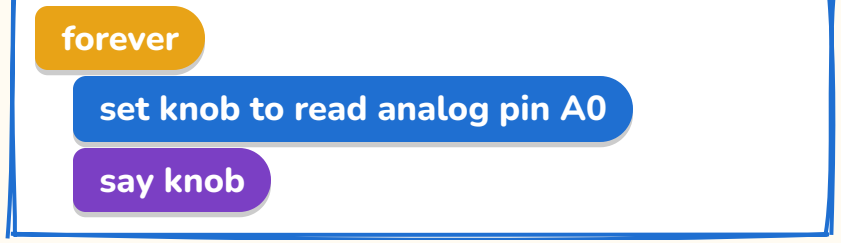
# Read your knob

| Knob position | Our reading | Prediction |
|---|---|---|
| All the way left | | 0? |
| Half way | | |
| All the way right | | 1023? |
| Where you think 250 is | | |



**knob (10k)**

Smooth?
Does the number go up smoothly, or jump? Turn slowly and watch.




**Live mode**

In mBlock choose the **Arduino Uno** device and switch to **Live**. The sprite can then read the knob straight away!

**Wiring:** outer legs to **5V** and **GND**, middle leg to **A0**. Backwards? Swap the outer legs!

Class game

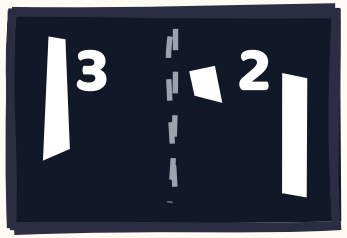
# Human Pong

## How to play

1. Two players sit at the ends of a desk, each holding a notebook as a paddle.
2. Gently bat a scrunched-up paper ball across the desk.
3. The class are the **computer**: shout the rule every time something happens.
4. **"Wall: bounce!" · "Paddle: bounce back!" · "Missed: point!"**
5. First to 5 points wins. Swap players.

## Then discuss

Which rules did the 'computer' need to know? Write them as **if … then …** sentences.




Big idea

A video game is just a list of simple rules checked again and again, very fast.

Build it

# Our Pong kit (per group)



**Arduino Uno**
The controller's brain



**Potentiometer**
The paddle knob



**Push button**
Serve the ball (+ 10k)



**LED + 220 Ω**
Lights up on a point



**Buzzer**
Beeps on a bounce



**Laptop**
mBlock 5 + the game

Also: breadboard and jumper wires. Put the knob and button in a small box or cup: now it is a real controller!

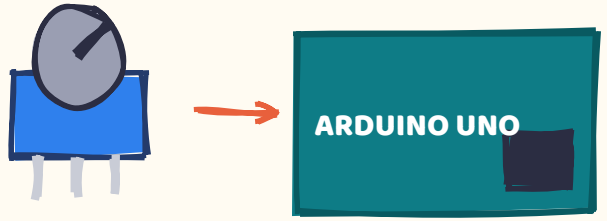
**Scratch first**
No Arduino? Build the game in Scratch with arrow keys first, then add the knob.

MAKE

Objective 1 · Knob to number · Build step 1 of 4

# Wire the controller

| Part | Connect to |
|---|---|
| Knob outer legs | 5V and GND |
| Knob middle leg | A0 |
| Button | pin 2 and 5V |
| 10k resistor | pin 2 to GND (pull-down) |
| LED + 220 Ω | pin 12 to GND |
| Buzzer + | pin 11 |




knob (10k)

**Why the 10k?**

Without it, pin 2 'floats' and reads random 0s and 1s. The 10k gently holds it at **0** until you press.

Controller box

Poke the knob through a paper cup. Hold it like a real game controller!

Objective 2 · Build step 2 of 4

# Make the paddle follow the knob

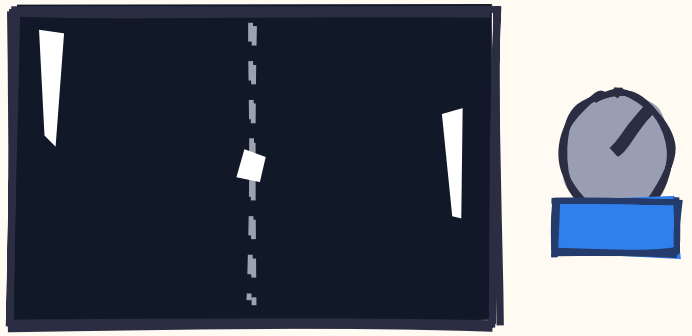
**a** Draw a **Paddle** sprite: a thin rectangle, about 10 × 70 pixels.

**b** Set its x to **-220** (left side of the stage).

**c** Add: **forever** → **set y to (knob ÷ 1023 × 300 - 150)**.

**d** Turn the knob slowly. Does the paddle reach the top AND the bottom?

**e** Paddle moves the wrong way? Change the formula to **150 - knob ÷ 1023 × 300**.

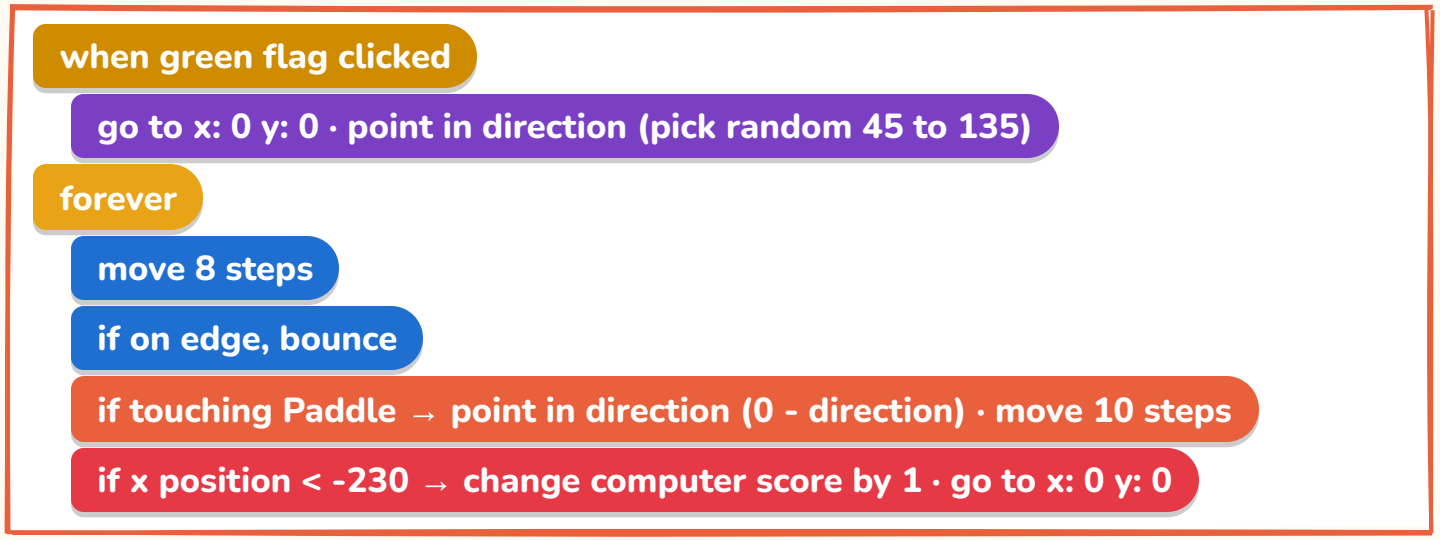


**Check:** Turning the knob moves the paddle smoothly from top to bottom.

Use **Live** mode, not Upload: the sprites live on the laptop.

Objective 3 · Game loop · Build step 3 of 4

# Code the ball




### The other side

Draw a wall or a computer paddle on the right that always bounces the ball back. Then you play alone against the computer!

Stuck to the paddle?
Add move 10 steps after the bounce so the ball escapes before the next check.

Objective 3 · Build step 4 of 4

# Serve, score and sound

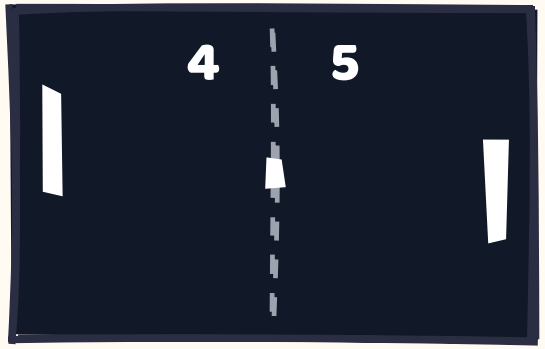
**a** Make two variables: **my score** and **computer score**. Tick them to show on the stage.

**b** Ball waits in the middle until the **button** on pin 2 is pressed. Then it serves!

**c** On a bounce off your paddle: **beep** the buzzer on pin 11.

**d** When you score: light the **LED** on pin 12 for 1 second.

**e** First to **5** wins: show a 'You win!' or 'Try again' message.




**Check:** The button serves, the score changes and the game ends at 5.

Objective 4 · Playtest

# Playtest like a game designer

## The idea

Game designers watch **real players** try their game. They do not explain anything: they watch where players get stuck or bored, then improve the game.

Swap seats with another team. Play their game for **3 minutes**, then fill in their feedback card.

| Feedback card | Answer |
|---|---|
| Fun? (1 to 5) | |
| Too easy or too hard? | |
| One thing I loved | |
| One thing to change | |
| Our change after feedback | |

**Example:** testers said "the ball is too slow and boring". The team made the ball speed up by 1 step after every bounce. Now every rally gets more exciting!

Level up!

# Game designer challenges

## Choose your challenge



**Two players:** add a second knob on A1 for a friend's paddle on the right.



**Speed up:** the ball gets 1 step faster after every bounce.



**Power-up:** a star appears; hit it to make your paddle bigger for 10 seconds.



**Sound design:** different beeps for walls, paddles and points.

## Think like an engineer

Why do you think a knob was better than buttons for the first Pong machines?

**Boss level**

Make the bounce angle depend on WHERE the ball hits the paddle, like real Pong.

Objective check

# Can I really do it? Prove it.

| | | |
|---|---|---|
| 01 | Turn your knob and read out 3 numbers. What are the smallest and biggest possible? | ○ ○ ○ |
| 02 | Explain the paddle formula. What y does the paddle get when the knob reads 1023? | ○ ○ ○ |
| 03 | Point to the **if** rule that makes the ball bounce off your paddle. | ○ ○ ○ |
| 04 | Tell us one change you made after another team playtested your game. | ○ ○ ○ |

**Rate yourself**

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

**Teacher:** Hold a mini tournament: winners of each table play each other on the projector.

Show what you know

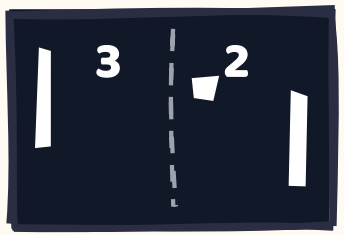
# What did we learn?

- [x] A potentiometer turns a twist into 0 to 1023
- [x] Mapping changes one range into another
- [x] A game loop moves, checks rules and repeats
- [x] Bouncing is just an if rule
- [x] Playtesting makes games better

**Exit ticket:** the knob reads 0. Where is the paddle, and why?

**At home**

Teach someone at home how to play your Pong game. What did they find hard?




**Next class:** build an AI Recycling Sorter!